Fitness Tracking Management System

Software Requirements Specification

Database Systems Course Project

Version	1.0
Year	2026
Document Type	Formal SRS Report

Prepared as a database-centered full-stack web application specification

Table of Contents

Table of contents will update when opened in Word.

1. Project Overview

1.1 Project Background

The developer currently has a regular fitness habit, but each gym visit often begins with uncertainty about what exercises to perform, which body parts to train, and how to record weight and repetitions in a useful way. Searching online or asking AI every time is inconvenient and leaves workout records scattered across different tools.

This project therefore designs a database-centered fitness tracking system that integrates training goals, workout plan generation, active workout recording, historical workout review, training volume statistics, and body weight history. The system stores users, authentication data, body parts, exercises, workout plans, workout sessions, workout sets, and body records in a relational database.

1.2 Problem Statement

- Fitness beginners and general gym users often do not know which exercises to perform on a given day.
- Paper notes or phone memos are not structured enough for SQL queries, long-term analysis, or dashboard statistics.
- Without login and user ownership, the system cannot reliably preserve personal workout history.
- Exercises, body parts, plans, sessions, and sets have clear relationships, making the project suitable for relational database design.

1.3 Project Goals

- Provide registration, login, and logout so each user can manage personal fitness data.
- Design a normalized relational database with clear primary keys, foreign keys, one-to-many relationships, and many-to-many relationships.
- Support CRUD operations for user profiles, preferred body parts, exercises, workout plans, workout sessions, workout sets, and body records.
- Provide workout history, training volume, weekly workout count, most trained body part, most used exercise, and body weight history.
- Implement a Start Workout button that creates an in-progress workout session and saves each completed set immediately.
- Deploy the application and cloud database so the final system does not rely on localhost.

1.4 Scope

Version 1.0 includes login and registration, database CRUD, workout plan generation, active workout tracking, historical records, and dashboard statistics. The first recommendation engine is rule-based and uses database records. A future LLM integration can be added later, but the database remains the core of the project.

1.5 Email Validation Policy

The first version validates email format and enforces uniqueness through the user_auth.email UNIQUE constraint. It does not send verification emails in version 1.0 because email verification would require a third-party email service, verification tokens, token expiration, resend logic, and additional error handling. The database still reserves email_verified and email_verified_at for future extension.

2. System Architecture and Tech Stack

2.1 Recommended Architecture

The system uses a frontend, backend API, and cloud PostgreSQL database. The frontend handles forms, workout plan views, active workout input, and dashboard pages. The backend handles authentication, authorization, business rules, recommendation logic, and SQL access. PostgreSQL stores all structured data.

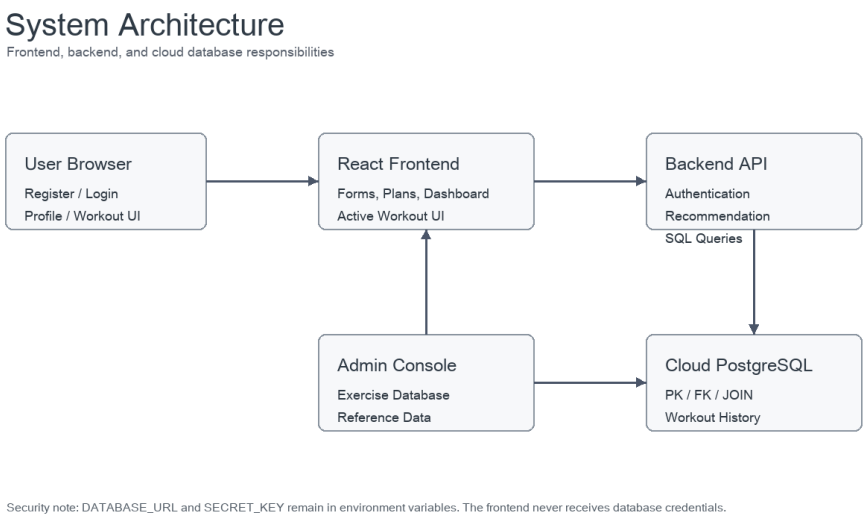


Figure 1. System Architecture

2.2 Suggested Tech Stack

Layer	Recommended Technology	Purpose
Frontend	React / Next.js / Vite	Build user interface, profile forms, workout plan pages, active workout pages, and dashboard views.
Backend	Node.js Express or Python FastAPI	Handle API routes, login, authorization, database access, and recommendation logic.
Database	PostgreSQL	Cloud relational database with FK, JOIN, GROUP BY, aggregate queries, and indexes.
ORM / Query	Prisma, SQLAlchemy, or raw SQL	Manage schema migration and application queries. Important course queries can remain as raw SQL examples.
Deployment	Render / Railway / Supabase / Neon / Vercel	Deploy a public web application and cloud database.
Security	bcrypt or argon2, JWT or secure session	Hash passwords and manage login state safely.

2.3 High Level Data Flow

- The user registers or logs in through the frontend.
- The backend validates credentials and creates a session or token.
- The user sets profile data, training goal, preferred body parts, weekly training days, and training duration.
- The backend queries exercises, body parts, and training goals to generate a workout plan.
- The user clicks Start Workout, and the backend creates a workout_sessions row.
- Each completed set is saved to workout_sets immediately.
- Dashboard queries use JOIN, GROUP BY, SUM, COUNT, and AVG to produce statistics.

3. User Flow

3.1 Main Flow

```
Enter website
  -> Register / Login
  -> Create or update user profile
  -> Select training goal
  -> Select preferred body parts
  -> Select training days and workout duration
  -> Generate workout plan from database exercises and rule-based logic
  -> Review plan details and exercise references
  -> Click Start Workout
  -> Create workout_sessions row with status = in_progress
  -> Save each completed set into workout_sets
  -> Complete workout and update session status
  -> Review Workout History, Training Volume, and Body Weight History
```

3.2 Start Workout Button Behavior

- The user opens a plan day and clicks Start Workout.
- The backend creates one workout_sessions row with user_id and optional plan_day_id.
- The initial session status is in_progress.
- The active workout page shows exercises from plan_exercises.
- Each saved set inserts one row into workout_sets with exercise_id, set_number, weight_kg, and reps.
- When the user finishes, workout_sessions.status changes to completed and ended_at is recorded.
- If the user leaves midway, the in_progress session can be resumed.

3.3 Roles

Role	Description	Main Permissions
Guest	Unauthenticated visitor	View landing page, register, and login.
User	General fitness user	Manage own profile, goals, plans, sessions, sets, and body records.
Admin	System maintainer	Maintain exercise database, body parts, training goals, and reference

Role	Description	Main Permissions
		data.

4. Functional Requirements

ID	Feature	Requirement
FR-01	Register	Users can create an account with name, email, and password. The system checks email format, uniqueness, and password strength.
FR-02	Login and Logout	Users can log in and log out. The system creates and revokes user sessions.
FR-03	Profile Management	Users can create and update age, gender, height, weight, and training experience.
FR-04	Training Goal	Users can select Muscle Gain, Fat Loss, Strength, or General Fitness.
FR-05	Preferred Body Parts	Users can select multiple body parts through the user_body_parts junction table.
FR-06	Exercise Database	The system stores exercise name, body part, difficulty, equipment, description, and image URL.
FR-07	Generate Workout Plan	The system generates workout_plans, plan_days, and plan_exercises from user preferences and database exercises.
FR-08	View Workout Plan	Users can view day title, exercises, target sets, target reps, rest time, and image reference.
FR-09	Start Workout	Users can create an in-progress workout session from a plan day.
FR-10	Record Sets	Users can record exercise, set number, weight, reps, and completion time.
FR-11	Workout History	Users can query previous workouts with exercises, body parts, sets, weight, reps, and volume.
FR-12	Dashboard	The system shows total volume, weekly workout count, most trained body part, most used exercise, and weight trend.
FR-13	Body Records	Users can add and review body weight records.

ID	Feature	Requirement
FR-14	Search	Users can search exercises, body parts, and workout history.
FR-15	Admin Management	Admins can create, update, and deactivate exercise and reference data.

4.1 CRUD Mapping

Entity	Create	Read	Update	Delete
users	Register	Profile	Edit profile	Deactivate account
training_goals	Admin create	Goal options	Admin edit	Admin deactivate
body_parts	Admin create	Body part list	Admin edit	Admin deactivate
user_body_parts	Select preferences	View preferences	Replace preferences	Remove preference
exercises	Admin create	Exercise search	Admin edit	Admin deactivate
workout_plans	Generate plan	Plan detail	Regenerate or rename	Delete plan
workout_sessions	Start workout	Workout history	Complete or cancel	Delete own session
workout_sets	Save set	Set history	Edit weight or reps	Delete set
body_records	Add record	Weight history	Edit record	Delete record

5. Database Design and ER Diagram

5.1 Database Design Principles

- All important data is stored in PostgreSQL and not only in localStorage or a local database.
- Every major table has a primary key.
- Cross-table relationships use foreign keys to maintain referential integrity.
- Many-to-many relationships use junction tables, such as user_body_parts.
- Multiple body parts are not stored as a comma-separated VARCHAR field.
- Authentication data is separated into user_auth to avoid mixing password_hash with profile data.
- User-owned records include user_id so queries and updates can be scoped to the logged-in user.

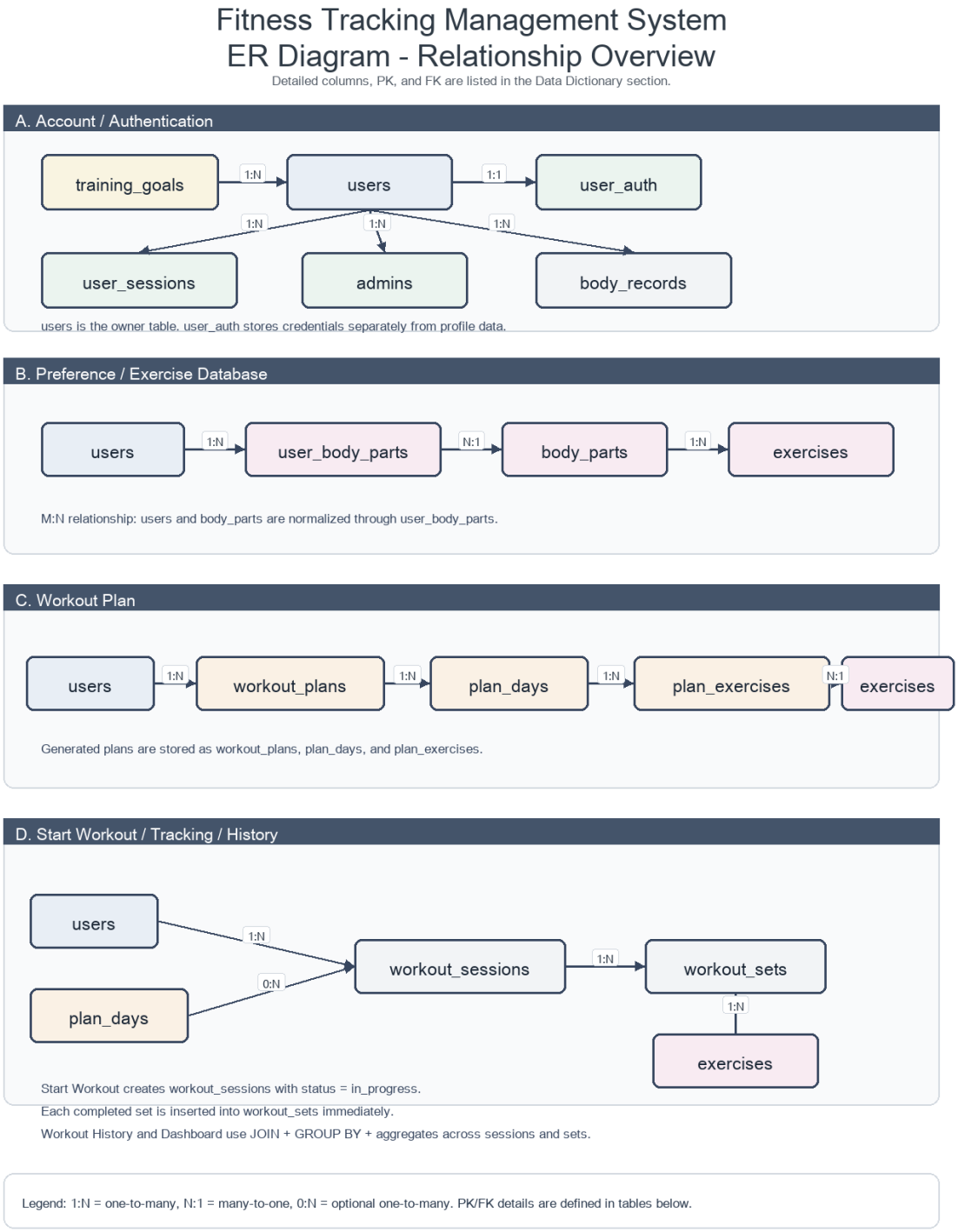


Figure 2. Entity Relationship Diagram

5.2 Entities Overview

Table	Purpose
users	Stores profile and user-owned settings.
user_auth	Stores login email, password_hash, and email verification fields.
user_sessions	Stores session token hash, expiration, and revoked time.

Table	Purpose
training_goals	Stores training goal reference data.
body_parts	Stores body part reference data.
user_body_parts	Junction table between users and body_parts.
exercises	Stores exercise name, body part, difficulty, equipment, description, and image URL.
workout_plans	Stores generated training plans.
plan_days	Stores day number and day focus in a workout plan.
plan_exercises	Stores exercises assigned to each plan day.
workout_sessions	Stores actual workout sessions created by Start Workout.
workout_sets	Stores every completed set with weight and reps.
body_records	Stores body weight and body composition records.
admins	Stores admin role assignments.

5.3 Relationships

Relationship	Explanation
users 1:1 user_auth	Each user has one credential record.
users 1:N user_sessions	One user can have many login sessions.
training_goals 1:N users	Many users can select the same training goal.
users M:N body_parts	Resolved by user_body_parts.
body_parts 1:N exercises	One body part can have many exercises.
users 1:N workout_plans	One user can generate many plans.
workout_plans 1:N plan_days	One plan contains multiple training days.
plan_days 1:N plan_exercises	One training day contains multiple planned exercises.
exercises 1:N plan_exercises	One exercise can appear in many plans.
users 1:N workout_sessions	One user can complete many workout sessions.
workout_sessions 1:N workout_sets	One workout session contains many set records.
exercises 1:N workout_sets	One exercise can have many actual set records.
users 1:N body_records	One user can have many body records.

6. Data Dictionary

users

Column	Data Type	Constraint	Description
user_id	BIGSERIAL	PK	System-generated user ID.
name	VARCHAR(100)	NOT NULL	User display name.
gender	VARCHAR(20)	NULL	Optional gender.
age	INT	CHECK 10 to 100	Age.
height_cm	NUMERIC(5,2)	CHECK > 0	Height in centimeters.
weight_kg	NUMERIC(5,2)	CHECK > 0	Current body weight.
training_experience	VARCHAR(20)	CHECK Beginner / Intermediate / Advanced	Training experience level.
training_goal_id	BIGINT	FK -> training_goals	Main training goal.
training_days_per_week	INT	CHECK 2 to 6	Weekly training frequency.
training_duration_minutes	INT	CHECK 30 / 60 / 90	Expected duration per workout.
created_at	TIMESTAMP	DEFAULT now()	Creation time.
updated_at	TIMESTAMP	DEFAULT now()	Last update time.

user_auth

Column	Data Type	Constraint	Description
auth_id	BIGSERIAL	PK	Authentication record ID.
user_id	BIGINT	FK UNIQUE -> users	One-to-one user reference.
email	VARCHAR(255)	NOT NULL UNIQUE	Login email.
password_hash	VARCHAR(255)	NOT NULL	Hashed password only.
email_verified	BOOLEAN	DEFAULT FALSE	Reserved for future email verification.
email_verified_at	TIMESTAMP	NULL	Future verification timestamp.
last_login_at	TIMESTAMP	NULL	Last login time.
created_at	TIMESTAMP	DEFAULT now()	Creation time.

user_sessions

Column	Data Type	Constraint	Description
session_id	BIGSERIAL	PK	Session ID.
user_id	BIGINT	FK -> users	Session owner.
session_token_hash	VARCHAR(255)	UNIQUE NOT NULL	Hashed session token.

Column	Data Type	Constraint	Description
expires_at	TIMESTAMP	NOT NULL	Expiration time.
revoked_at	TIMESTAMP	NULL	Logout or revocation time.
created_at	TIMESTAMP	DEFAULT now()	Creation time.

training_goals

Column	Data Type	Constraint	Description
training_goal_id	BIGSERIAL	PK	Training goal ID.
goal_name	VARCHAR(50)	UNIQUE NOT NULL	Muscle Gain, Fat Loss, Strength, or General Fitness.
description	TEXT	NULL	Goal description.

body_parts

Column	Data Type	Constraint	Description
body_part_id	BIGSERIAL	PK	Body part ID.
body_part_name	VARCHAR(50)	UNIQUE NOT NULL	Chest, Back, Shoulders, Legs, Core, and related body parts.
description	TEXT	NULL	Body part description.

user_body_parts

Column	Data Type	Constraint	Description
user_id	BIGINT	PK, FK -> users	User ID.
body_part_id	BIGINT	PK, FK -> body_parts	Preferred body part ID.
priority	INT	DEFAULT 1	Preference priority.
created_at	TIMESTAMP	DEFAULT now()	Creation time.

exercises

Column	Data Type	Constraint	Description
exercise_id	BIGSERIAL	PK	Exercise ID.
exercise_name	VARCHAR(120)	UNIQUE NOT NULL	Exercise name.
body_part_id	BIGINT	FK -> body_parts	Primary trained body part.
difficulty_level	VARCHAR(20)	CHECK Beginner / Intermediate / Advanced	Exercise difficulty.
equipment	VARCHAR(100)	NULL	Equipment.

Column	Data Type	Constraint	Description
description	TEXT	NULL	Exercise description.
image_url	TEXT	NULL	Reference image URL.
is_active	BOOLEAN	DEFAULT TRUE	Whether the exercise is active.

workout_plans

Column	Data Type	Constraint	Description
workout_plan_id	BIGSERIAL	PK	Workout plan ID.
user_id	BIGINT	FK -> users	Plan owner.
plan_name	VARCHAR(120)	NOT NULL	Plan name.
training_goal_id	BIGINT	FK -> training_goals	Goal used to generate plan.
days_per_week	INT	CHECK 2 to 6	Training days per week.
duration_minutes	INT	CHECK 30 / 60 / 90	Workout duration.
generated_by	VARCHAR(30)	DEFAULT rule_based	Recommendation source.
created_at	TIMESTAMP	DEFAULT now()	Creation time.

plan_days

Column	Data Type	Constraint	Description
plan_day_id	BIGSERIAL	PK	Plan day ID.
workout_plan_id	BIGINT	FK -> workout_plans	Parent workout plan.
day_number	INT	NOT NULL	Day order.
day_title	VARCHAR(120)	NOT NULL	Training focus title.
focus_notes	TEXT	NULL	Notes for the day.

plan_exercises

Column	Data Type	Constraint	Description
plan_exercise_id	BIGSERIAL	PK	Planned exercise ID.
plan_day_id	BIGINT	FK -> plan_days	Parent plan day.
exercise_id	BIGINT	FK -> exercises	Exercise.
exercise_order	INT	NOT NULL	Display order.
target_sets	INT	NOT NULL	Recommended sets.
target_reps	VARCHAR(30)	NOT NULL	Recommended reps.
target_weight_note	VARCHAR(120)	NULL	Weight guidance.

Column	Data Type	Constraint	Description
rest_seconds	INT	DEFAULT 60	Rest time.

workout_sessions

Column	Data Type	Constraint	Description
workout_session_id	BIGSERIAL	PK	Actual workout session ID.
user_id	BIGINT	FK -> users	Session owner.
workout_plan_id	BIGINT	FK -> workout_plans NULL	Optional source plan.
plan_day_id	BIGINT	FK -> plan_days NULL	Optional source plan day.
session_date	DATE	DEFAULT CURRENT_DATE	Workout date.
started_at	TIMESTAMP	DEFAULT now()	Start time.
ended_at	TIMESTAMP	NULL	End time.
status	VARCHAR(20)	CHECK in_progress / completed / cancelled	Session status.
notes	TEXT	NULL	Workout notes.

workout_sets

Column	Data Type	Constraint	Description
workout_set_id	BIGSERIAL	PK	Set record ID.
workout_session_id	BIGINT	FK -> workout_sessions	Parent workout session.
exercise_id	BIGINT	FK -> exercises	Exercise performed.
set_number	INT	NOT NULL	Set number.
weight_kg	NUMERIC(6,2)	CHECK >= 0	Weight in kilograms.
reps	INT	CHECK > 0	Repetitions.
volume	NUMERIC(10,2)	computed	weight_kg * reps.
completed_at	TIMESTAMP	DEFAULT now()	Completion time.

body_records

Column	Data Type	Constraint	Description
body_record_id	BIGSERIAL	PK	Body record ID.
user_id	BIGINT	FK -> users	Record owner.
record_date	DATE	NOT NULL	Record date.
weight_kg	NUMERIC(5,2)	CHECK > 0	Body weight.
body_fat_percentage	NUMERIC(5,2)	NULL	Optional body fat percentage.

Column	Data Type	Constraint	Description
notes	TEXT	NULL	Notes.
created_at	TIMESTAMP	DEFAULT now()	Creation time.

admins

Column	Data Type	Constraint	Description
admin_id	BIGSERIAL	PK	Admin ID.
user_id	BIGINT	FK UNIQUE -> users	Admin user.
role	VARCHAR(50)	DEFAULT admin	Admin role.
created_at	TIMESTAMP	DEFAULT now()	Creation time.

7. Primary Keys Foreign Keys Relationships and 3NF

7.1 Primary Keys

- Major entity tables use BIGSERIAL surrogate primary keys.
- user_body_parts uses a composite primary key of user_id and body_part_id.
- workout_sets uses workout_set_id so each set can be edited or deleted independently.

7.2 Foreign Keys

```
user_auth.user_id               -> users.user_id
user_sessions.user_id           -> users.user_id
users.training_goal_id          -> training_goals.training_goal_id
user_body_parts.user_id         -> users.user_id
user_body_parts.body_part_id    -> body_parts.body_part_id
exercises.body_part_id          -> body_parts.body_part_id
workout_plans.user_id           -> users.user_id
workout_plans.training_goal_id  -> training_goals.training_goal_id
plan_days.workout_plan_id       -> workout_plans.workout_plan_id
plan_exercises.plan_day_id      -> plan_days.plan_day_id
plan_exercises.exercise_id      -> exercises.exercise_id
workout_sessions.user_id        -> users.user_id
workout_sessions.workout_plan_id -> workout_plans.workout_plan_id
workout_sessions.plan_day_id    -> plan_days.plan_day_id
workout_sets.workout_session_id -> workout_sessions.workout_session_id
workout_sets.exercise_id        -> exercises.exercise_id
body_records.user_id            -> users.user_id
admins.user_id                  -> users.user_id
```

7.3 Constraints and Indexes

Index	Reason
idx_user_auth_email	Speeds up login by email.
idx_exercises_body_part_id	Speeds up exercise filtering by body part.
idx_workout_sessions_user_date	Speeds up workout history by user and date.

Index	Reason
idx_workout_sets_session_id	Speeds up set lookup for one workout session.
idx_body_records_user_date	Speeds up body weight history queries.
idx_plan_days_plan_day_number	Speeds up plan day ordering.

7.4 3NF Normalization

- First Normal Form is satisfied because fields store atomic values. Multiple body parts are stored in user_body_parts, not one comma-separated field.
- Second Normal Form is satisfied because non-key attributes in junction tables depend on the full key.
- Third Normal Form is satisfied because training goals, body parts, exercises, authentication, sessions, and body records are separated by subject.
- Authentication data is stored in user_auth instead of mixing password_hash with profile attributes.
- Actual workout records are separated into workout_sessions and workout_sets to avoid repeated session data.

8. SQL Query Requirements

8.1 Create Insert Example

```
INSERT INTO users
  (name, gender, age, height_cm, weight_kg, training_experience,
   training_goal_id, training_days_per_week, training_duration_minutes)
VALUES
  ('Loise', 'Female', 21, 160, 50, 'Beginner', 1, 4, 60);
```

8.2 Read with JOIN Workout History

```
SELECT
  ws.session_date,
  e.exercise_name,
  bp.body_part_name,
  wset.set_number,
  wset.weight_kg,
  wset.reps,
  (wset.weight_kg * wset.reps) AS set_volume
FROM workout_sessions ws
JOIN workout_sets wset
  ON ws.workout_session_id = wset.workout_session_id
JOIN exercises e
  ON wset.exercise_id = e.exercise_id
JOIN body_parts bp
  ON e.body_part_id = bp.body_part_id
WHERE ws.user_id = :user_id
ORDER BY ws.session_date DESC, e.exercise_name, wset.set_number;
```

8.3 GROUP BY and Aggregate Weekly Training Volume

```
SELECT
  DATE_TRUNC('week', ws.session_date) AS training_week,
  SUM(wset.weight_kg * wset.reps) AS weekly_volume,
  COUNT(DISTINCT ws.workout_session_id) AS workout_count
FROM workout_sessions ws
JOIN workout_sets wset
  ON ws.workout_session_id = wset.workout_session_id
WHERE ws.user_id = :user_id
  AND ws.status = 'completed'
GROUP BY DATE_TRUNC('week', ws.session_date)
ORDER BY training_week DESC;
```

8.4 Most Trained Body Part

```
SELECT
  bp.body_part_name,
  COUNT(*) AS total_sets,
  SUM(wset.weight_kg * wset.reps) AS total_volume
FROM workout_sets wset
JOIN workout_sessions ws
  ON ws.workout_session_id = wset.workout_session_id
JOIN exercises e
  ON e.exercise_id = wset.exercise_id
JOIN body_parts bp
  ON bp.body_part_id = e.body_part_id
WHERE ws.user_id = :user_id
GROUP BY bp.body_part_name
ORDER BY total_sets DESC;
```

8.5 Body Weight History

```
SELECT record_date, weight_kg
FROM body_records
WHERE user_id = :user_id
ORDER BY record_date ASC;
```

8.6 Search Exercises

```
SELECT
  e.exercise_id,
  e.exercise_name,
  bp.body_part_name,
  e.difficulty_level,
  e.equipment
FROM exercises e
JOIN body_parts bp
  ON bp.body_part_id = e.body_part_id
WHERE LOWER(e.exercise_name) LIKE LOWER('%' || :keyword || '%')
   OR LOWER(bp.body_part_name) LIKE LOWER('%' || :keyword || '%')
ORDER BY e.exercise_name;
```

9. Workout Recommendation Algorithm

9.1 Algorithm Goal

Version 1.0 does not depend on an AI API. It uses database records and a rule-based recommendation algorithm to generate workout plans from training goal, training experience, preferred body parts, training days per week, and training duration.

9.2 Rule Based Logic

- Training experience determines sets, reps, rest seconds, and exercise difficulty.
- Beginner users receive beginner exercises and a smaller number of intermediate exercises.
- Muscle Gain uses moderate reps and higher volume. Strength uses lower reps and longer rest. Fat Loss uses higher reps and shorter rest. General Fitness uses moderate volume.
- days_per_week determines the split, such as full body, push-pull-legs, upper-lower, or focused body-part split.
- Preferred body parts appear more often, but other major muscle groups are still included.
- duration_minutes limits the number of exercises in each plan day.

9.3 Pseudocode

```
Input: user_id

1. Read user profile, training goal, experience, days_per_week, and duration_minutes.
2. Read preferred body parts from user_body_parts.
3. Build weekly split according to days_per_week.
4. Add preferred body parts to the split with higher frequency.
5. For each day:
   a. Query candidate exercises by body_part_id and difficulty_level.
   b. Select exercises by preference, difficulty, and equipment availability.
   c. Decide target sets, reps, and rest time based on goal and experience.
   d. Insert plan_days and plan_exercises.
6. Return workout_plan_id and detailed plan.
```

9.4 Example Output

Day	Focus	Exercises
Day 1	Chest + Triceps	Bench Press 3x10, Chest Press 3x12, Triceps Pushdown 3x12
Day 2	Back + Biceps	Lat Pulldown 3x10, Seated Row 3x12, Bicep Curl 3x12
Day 3	Legs + Core	Squat 3x10, Leg Press 3x12, Plank 3 sets
Day 4	Chest + Back	Incline Press 3x10, Lat Pulldown 3x10, Cable Row 3x12

9.5 Backend Validation

- Confirm that user_id exists and belongs to the logged-in user.
- Confirm that selected body parts exist in body_parts.

- Confirm that chosen exercises are active exercises.
- Confirm that target sets, reps, and rest_seconds are reasonable.
- Return a clear error or fallback selection if not enough candidate exercises exist.

10. Non Functional Requirements

Category	Requirement
Security	Passwords must be hashed with bcrypt or argon2. Users can only access their own data.
Privacy	Email, profile data, workouts, and body records must not be exposed to other users.
Performance	Common query fields need indexes, including email, user_id, session_date, and body_part_id.
Reliability	Each completed set is saved immediately to reduce the risk of losing workout data.
Maintainability	Schema migrations, seed data, README, and .env.example must be clear.
Deployment	Source code is stored on GitHub. The application and PostgreSQL database are deployed in the cloud.
Environment Variables	DATABASE_URL, SECRET_KEY, API keys, and passwords must not be committed to GitHub.
Responsive UI	The active workout page must work well on both mobile and desktop devices.

10.1 Environment Variables

```
DATABASE_URL=postgresql://USER:PASSWORD@HOST:PORT/DATABASE
SECRET_KEY=replace-with-random-secret
JWT_EXPIRES_IN=7d
OPENAI_API_KEY=optional-future-llm-key
NODE_ENV=development
```

10.2 GitHub Safety Rules

- .env must not be committed.
- .env.example must be provided as a safe template.
- DATABASE_URL, SECRET_KEY, API keys, and passwords must not be hard-coded.
- README should explain environment variables without revealing real secrets.

11. Acceptance Test Cases

ID	Scenario	Input or Action	Expected Result
AT-01	Register	Enter valid name, email, and password.	Create users and user_auth. Password is stored as hash.

ID	Scenario	Input or Action	Expected Result
AT-02	Duplicate Email	Register with an existing email.	System rejects the request.
AT-03	Login	Enter correct email and password.	Create a valid session and open dashboard.
AT-04	Profile	Edit age, height, weight, and experience.	users row is updated.
AT-05	Preferred Body Parts	Select Chest and Back.	Two user_body_parts rows are created.
AT-06	Generate Plan	Beginner, Muscle Gain, 4 days, Chest + Back.	Create workout_plans, plan_days, and plan_exercises.
AT-07	View Plan	Open plan page.	Show day title, exercise, sets, reps, rest, and image reference.
AT-08	Start Workout	Click Start Workout.	Create workout_sessions with status in_progress.
AT-09	Save Set	Enter Bench Press 60 kg x 12.	Create workout_sets and calculate set volume.
AT-10	Complete Workout	Click Finish Workout.	Set session status to completed and save ended_at.
AT-11	Workout History	Open history page.	JOIN query shows date, exercise, body part, sets, weight, reps, and volume.
AT-12	Training Volume	Open dashboard.	GROUP BY and SUM show weekly training volume.
AT-13	Weight History	Add several body records.	Body weight history is shown in date order.
AT-14	Admin Exercise CRUD	Admin creates or edits exercise.	exercises table is updated.
AT-15	Authorization	User A requests User B data.	API rejects the request.

12. Deployment Plan

12.1 Required Deployment Outcome

- Push source code to a GitHub repository.
- Deploy the web application to a public URL.
- Allow the instructor to open the URL without a GitHub account.
- Deploy PostgreSQL to the cloud instead of relying on localhost.
- Manage all sensitive values through environment variables.

12.2 Suggested Cloud Setup

Component	Option	Notes
Frontend and Backend	Render Web Service or Railway	Suitable for a full-stack service.
Database	Render PostgreSQL, Neon, or Supabase	Provides a cloud DATABASE_URL.
Static Assets	Public folder or image_url	Exercise images can use public URLs in version 1.0.
GitHub	Loisekoy repository	Stores source code, README, schema, and migration files.

12.3 README Requirements

- Project Description
- Database Design
- ER Diagram
- Installation
- Environment Variables
- Local Development
- Deployment
- Live Demo

13. Database Course Highlights

Course Requirement	How the Project Demonstrates It
Primary Key	Every major table has a PK. user_body_parts uses a composite PK.
Foreign Key	Users, exercises, plans, sessions, sets, and body records are connected through FK constraints.
1:N Relationship	users -> workout_sessions and workout_sessions -> workout_sets.
M:N Relationship	users <-> body_parts through user_body_parts.
Normalization	Goals, body parts, exercises, authentication, sessions, and records are separated and satisfy 3NF.
CRUD	Users, exercises, plans, sessions, sets, and body records support create, read, update, and delete cases.
JOIN	Workout History joins sessions, sets, exercises, and body parts.
GROUP BY	Weekly volume, most trained body part, and most used exercise require GROUP BY.
Aggregate Functions	SUM, COUNT, and AVG support dashboard statistics.
Indexes	Indexes support login, history, and common filtering

Course Requirement	How the Project Demonstrates It
	queries.
Data Ownership	All personal records are scoped by user_id.

14. Conclusion

Fitness Tracking Management System is a database-centered full-stack web application for a Database Systems course project. Its core value is the relational database design that connects users, authentication, goals, body parts, exercises, workout plans, workout sessions, workout sets, and body records. The design demonstrates primary keys, foreign keys, one-to-many relationships, many-to-many relationships, normalization, CRUD, JOIN, GROUP BY, aggregate functions, indexing, and data ownership.

The final implementation can use this specification as the reference for schema design, API behavior, dashboard queries, deployment, and project documentation.